

Extending RANsacked for NAS-Aware and Stateful Fuzzing of 5G Control-Plane Interfaces

Name: Omkar Chavan
Roll No: CS24MTECH12013

Term Project Final Presentation
May 3, 2026



Project Recap

Problem Statement / Motivation

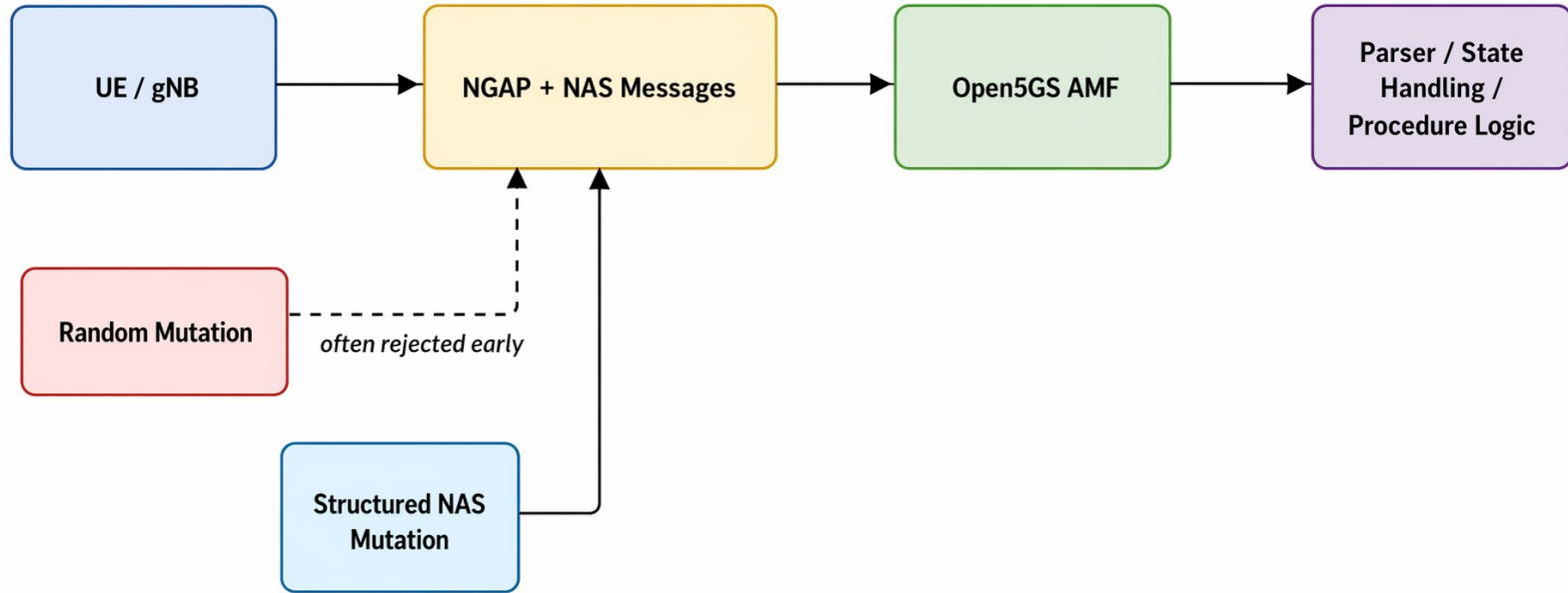
- NGAP and NAS are security-critical 5G control-plane interfaces.
- They handle externally influenced inputs early in the protocol flow.
- Structured malformed inputs can exercise deeper logic than random mutation.
- Prior work such as RANsacked [1] showed the value of structure-aware fuzzing, but deeper NAS-aware and selected procedure-level testing remain less explored.
- This motivates a trace-driven fuzzing workflow that mutates NAS fields inside realistic control-plane exchanges.

Key Objectives

- Build a trace-driven fuzzing workflow for 5G control-plane testing.
- Implement NAS-aware structured mutation inside realistic NGAP/NAS exchanges.
- Support selected procedure-level / state-aware mutation scenarios.
- Evaluate outcomes through live execution, campaign tracking, and reportable analysis.

Final focus: a working multi-message NAS-aware fuzzing prototype with live execution and reproducible campaign evidence.

Why NAS-Aware Mutation Matters



Structured malformed NAS inputs are more likely than random mutation to reach deeper parser, state-handling, and procedure logic.

Related Work and Project Positioning

Work / System	What it does	Gap addressed by our work
RANsacked [1]	Motivates structure-aware fuzzing for telecom control-plane protocols.	Extends this direction toward NAS-aware mutation and selected procedure-aware testing.
Optimizing 5G Protocol Fuzzing: Structure-Aware Techniques and Feedback Integration (ITASEC 2025) [2]	Implements structure-aware, feedback-guided 5G protocol fuzzing for deeper state reachability.	Adds a more reproducible, implementation-facing workflow with live Open5GS [3] execution, history tracking, and report export.
AFL++ / LibFuzzer [5, 6]	Strong generic coverage-guided fuzzing engines.	Adds the missing telecom-specific layer: NAS-aware modeling, procedure context, and campaign automation.

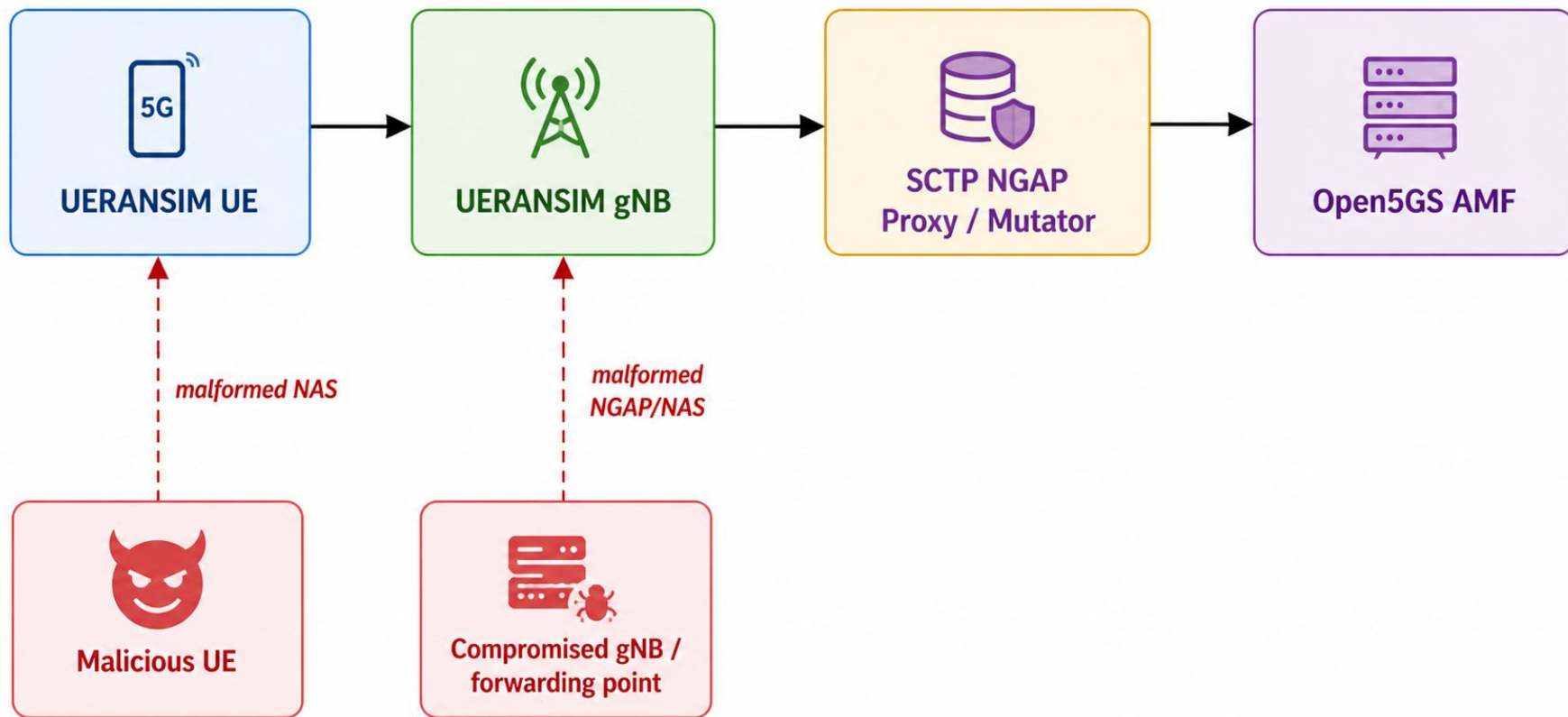
System and Threat Model

System Model

- **Target system:** Open5GS [3] AMF handling 5G control-plane signaling
- **Traffic source:** UERANSIM [4] UE and gNB in a software-only testbed
- **Mutation point:** custom SCTP NGAP proxy placed between gNB and AMF
- **Focus:** NGAP-carried NAS messages in early and selected later control-plane procedures

Threat Model

- **Malicious UE:** sends malformed or adversarial NAS messages toward the AMF
- **Compromised gNB / forwarding point:** injects malformed NGAP/NAS control-plane traffic
- **Goal:** observe parser, state-handling, and procedure-level behavior under structured malformed inputs



Threat Model Diagram

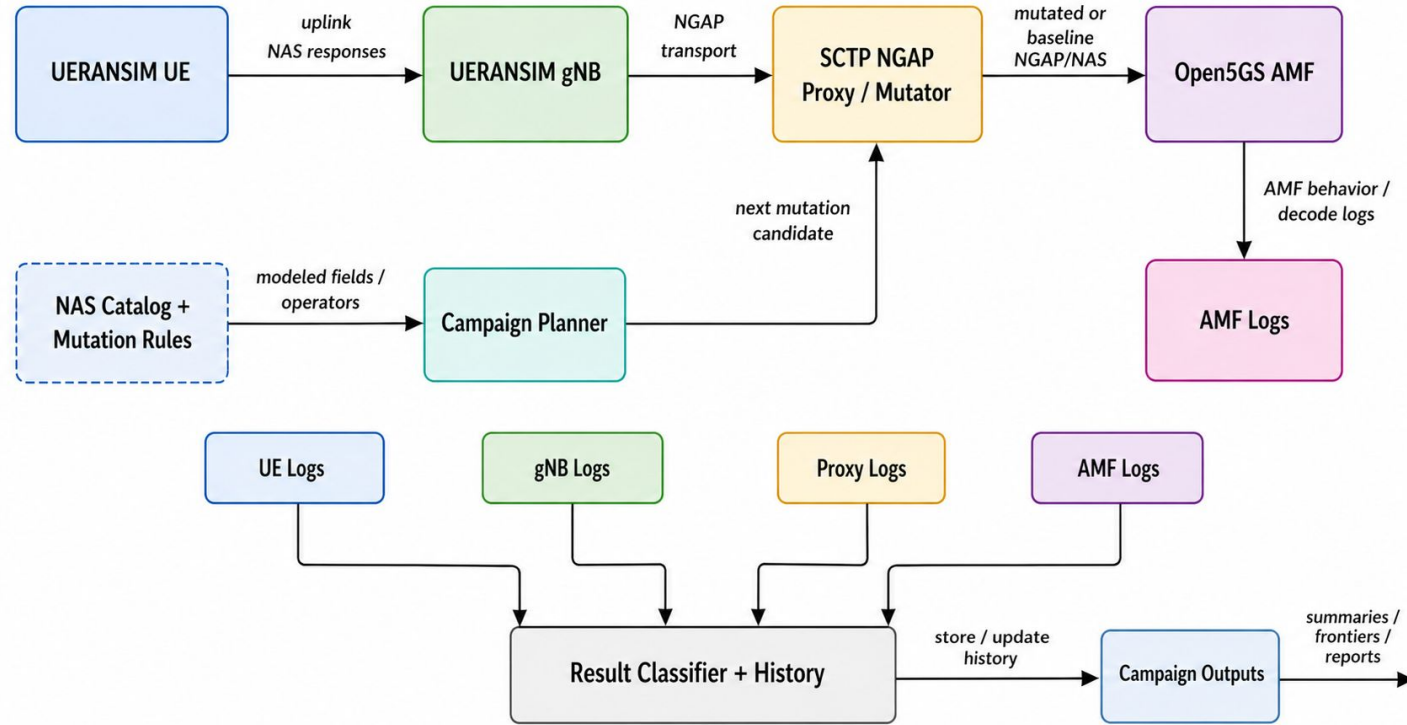
Key Assumptions

- Software-only evaluation using Open5GS [3] and UERANSIM [4]
- No SDR or over-the-air experiments in project scope
- Focus is robustness testing of control-plane handling, not radio-layer attacks

System Architecture

- UERANSIM UE and gNB generate realistic 5G control-plane traffic.
- A custom SCTP NGAP proxy sits between the gNB and Open5GS AMF.
- Python tooling selects mutation candidates, drives execution, and records outcomes.
- Logs from the AMF, proxy, gNB, and UE are used for classification and campaign reporting.

System Architecture Diagram

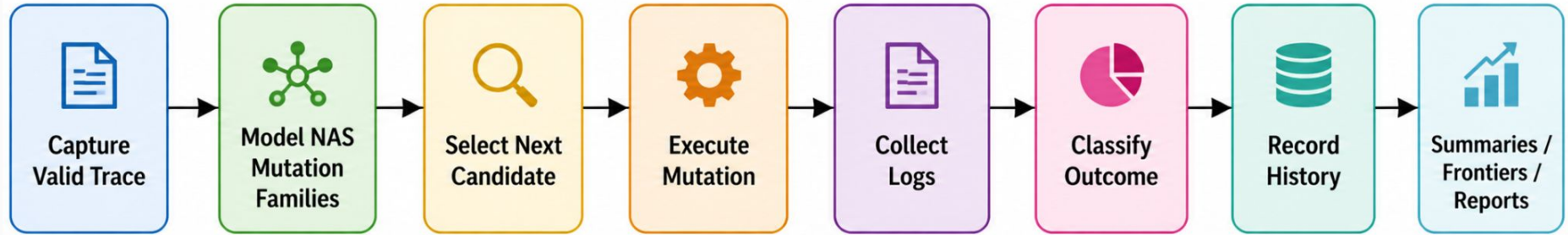


Live execution path and campaign-analysis pipeline used in the final freeze-state evaluation

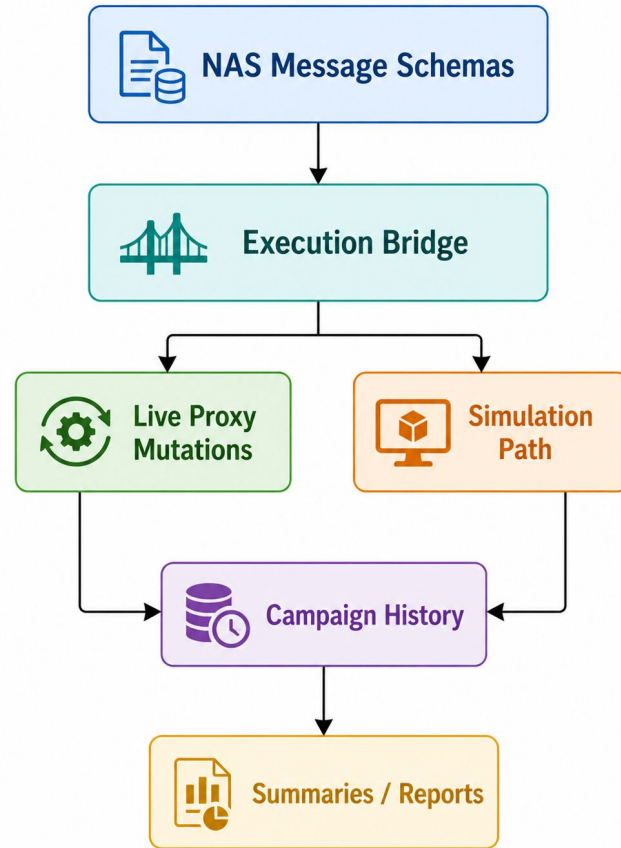
Key Algorithm / Core Logic

- Start from a valid baseline NGAP/NAS procedure trace
- Enumerate modeled mutation candidates for the selected NAS message or family
- Use campaign history to prioritize the next frontier
- Execute the case through live proxy mutation or trace-based simulation
- Classify the observed outcome from AMF, proxy, gNB, and UE logs
- Record the result and update summaries, frontiers, and reports

Workflow diagram



Implementation Details



The final implementation is not just a proxy, but a reusable NAS-aware campaign framework around Open5GS.

Software Stack

- Open5GS AMF as the target 5G core component
- UERANSIM UE and gNB for realistic control-plane traffic generation
- Custom SCTP NGAP proxy for live mutation injection
- Python-based campaign tooling for mutation planning, execution support, and reporting

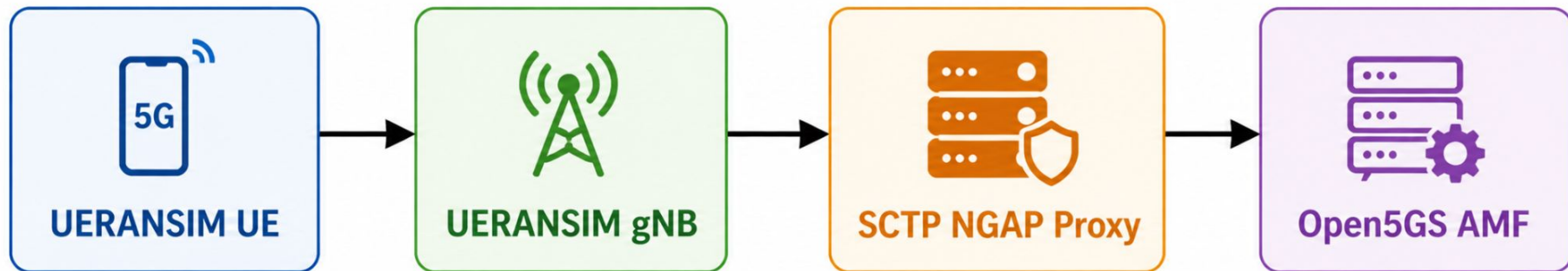
Core Components Built

- Schema-driven NAS mutation catalog for multiple message types
- Execution bridge from modeled operators to live proxy or simulation modes
- Campaign planner, history tracker, and frontier selection logic
- Log-based outcome classification and report export pipeline

Implementation Emphasis

- Strongest live support was first stabilized on Registration Request
- The same framework was then extended to Identity Response, Authentication Response, Security Mode Complete, and PDU Session Establishment Request
- The final system supports both live execution and simulation-backed analysis for selected cases

Experimental Setup



Setup

- **Target core:** Open5GS AMF
- **Traffic source:** UERANSIM UE and gNB
- **Mutation point:** custom SCTP NGAP proxy between gNB and AMF
- **Execution modes:** live proxy mutation and trace-based simulation

Reproducibility

- Campaign runs are driven through CLI-based planning, execution, recording, and summary generation
- Final freeze-state artifacts include submission reports and campaign summaries
- The workflow is reproducible in a software-only Open5GS [3] + UERANSIM [4] environment

Contd..

Metrics / Outcome Classes

- unexpected-accept
- early-semantic-reject
- deep-decoder-failure
- timeout-retry

Baseline(s) for Comparison

- Packet-level mutation inspired by RANsacked [1]
- Opaque-byte NAS mutation

Results and Analysis

Supported Coverage at Freeze State

- Modeled NAS messages: **5**
- Summarized mutation families: **24**
- Recorded campaign observations: **97**
- Live frontier exhausted for the current supported scope

Overall, field-aware and procedure-aware malformed inputs reached multiple layers of control-plane handling and produced both permissive and rejecting behaviors.

NAS MESSAGE MUTATION COVERAGE

	Message	Example Covered Families	
 1. REGISTRATION & IDENTITY PHASE	1 Registration Request	- Message type - Security header - Mobile identity	- NSSAI 5GMM capability
	2 Identity Response	- Message type - Security header	Identity payload corruption
 2. AUTHENTICATION PHASE	3 Authentication Response	- Message type - Security header	Authentication parameter corruption
 3. SECURITY ESTABLISHMENT PHASE	4 Security Mode Complete	- Message type - Security header inconsistency	- Wrong-state delivery - Nested IE corruption
 4. SESSION ESTABLISHMENT PHASE	5 PDU Session Establishment Request	Session IE corruption	Wrong-state delivery

Coverage Table

Key Results

Message	Strongest Covered Family	Main Observed Outcome	Final State
Registration Request	Mobile identity / Requested NSSAI / 5GMM capability	Unexpected accept, deep decoder failure, early semantic reject	Strong live coverage
Identity Response	Identity payload corruption	Unexpected accept	Likely saturated
Authentication Response	Authentication parameter corruption	Unexpected accept, early semantic reject, timeout retry	Fully explored
Security Mode Complete	Wrong-state delivery, nested IE corruption	Early semantic reject, unexpected accept, timeout retry	Strong live coverage
PDU Session Establishment Request	Session IE corruption, wrong-state delivery	Mixed live outcomes	Fully explored

Detailed family-wise results : [Mutation Families and Main Outcomes.docx](#)

Limitations and Failure Cases

Current Limitations

- The implementation is strongest for supported live proxy-executable NAS families, not for every modeled family.
- Some modeled Identity Response families remain planned-only, such as length inconsistency and wrong-state delivery.
- Broader NGAP mutation coverage and wider 5GSM / post-registration expansion remain future work.
- Execution is not yet fully generic for any modeled NAS field across all messages.

Failure / Outcome Patterns Observed

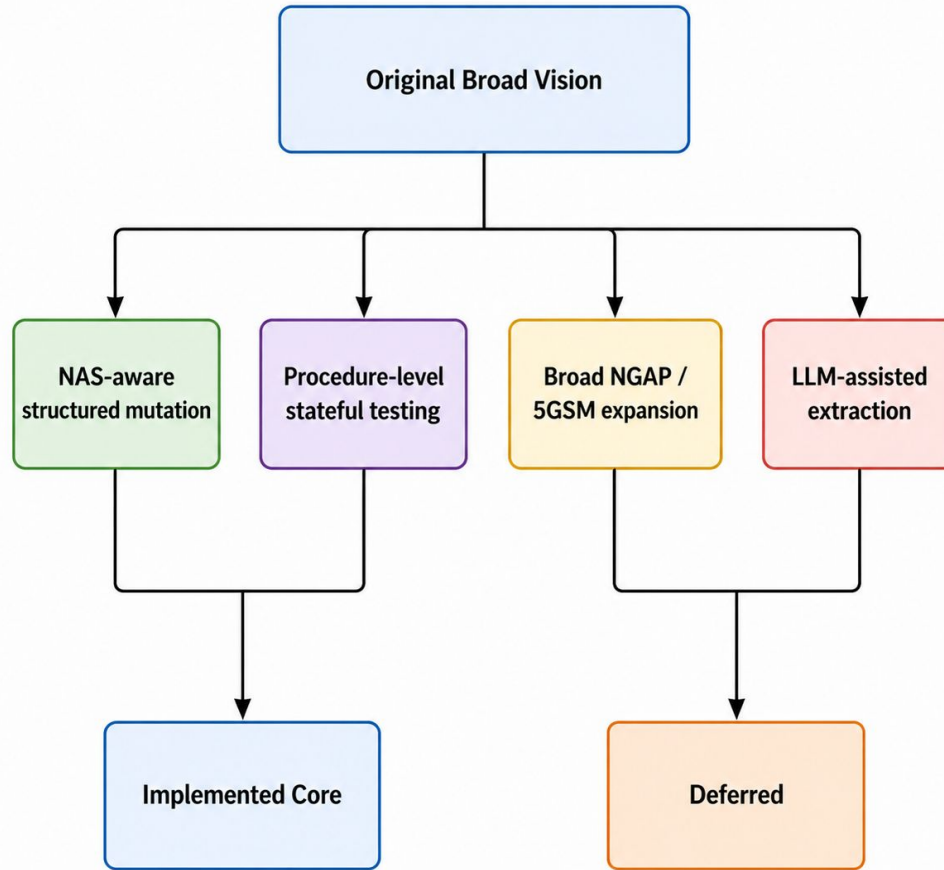
- Early semantic reject in some wrong-state or inconsistent-message cases
- Deep decoder failure in selected malformed but decodable message families
- Unexpected accept for several structured malformed NAS payload variations
- Timeout / retry behavior in some ambiguous or deeper-processing cases

Comparison with Objectives

Objective	Status	Evidence
Build a trace-driven fuzzing workflow for 5G control-plane testing	Completed	Working Open5GS + UERANSIM + proxy workflow with campaign planning, execution, classification, and reporting
Implement NAS-aware structured mutation inside realistic NGAP/NAS exchanges	Completed	Schema-driven NAS mutation catalog across 5 modeled NAS messages
Support selected procedure-level / state-aware mutation scenarios	Partially Completed	Wrong-state delivery and selected later-packet cases supported for Security Mode Complete and PDU Session Establishment Request

Objective	Status	Evidence
Evaluate behavior through live execution, history tracking, and reportable analysis	Completed	97 recorded observations, 24 mutation families, exported submission reports, final freeze-state summaries
Compare against simpler mutation baselines	Partially Completed	Qualitative comparison against packet-level mutation and opaque-byte NAS mutation; no full quantitative benchmark study

The main project objectives were achieved, with broader stateful coverage and full baseline comparison remaining partial.



docs.google.com/document/d/1ACK7SHbYry8YhMxvhHBaeiQ5XVnDPYp4beiBeSWajJM/edit?tab=t.qaxqmm7cfj7c

FINAL DEMO : Live TWiN Campaign Workflow

File Edit View Insert Format Tools Extensions Help

Normal text 100% Google... 13 B I U A

Document tabs

- FINAL DEMO 1
- FINAL DEMO 2

Demo Case

Message: Registration Request
Family: mobile-identity value corruption
Operator: toggle identity type bits inconsistently
Observed result: unexpected-accept

Registration Request (plain NAS)

Before mutation				
EPD	SecHdr	MsgType	Len	Mobile Identity Value
0x7e	0x00	0x41	0x000d	[0x01 [...] [payload]

After mutation				
EPD	SecHdr	MsgType	Len	Mobile Identity Value
0x7e	0x00	0x41	0x000d	[0x06 [...] [payload]

First mobile-identity payload byte changed: 0x01 -> 0x06

FINAL DEMO Documentation : Live TWiN Campaign Workflow

Work Distribution

Name	Core Responsibility	Key Tasks and Contributions
Omkar Chavan	End-to-end project ownership	Problem formulation, Open5GS + UERANSIM setup, SCTP NGAP proxy development, NAS mutation modeling, campaign tooling, experimentation, result analysis, and documentation

Conclusion and Future Work

Conclusion

- This project built a trace-driven, NAS-aware fuzzing workflow for 5G control-plane testing.
- The final system supports live proxy execution, campaign planning, classification, history tracking, and report generation.
- The evaluation reached 5 modeled NAS messages, 24 mutation families, and 97 recorded observations.
- Results show that structured malformed NAS inputs can trigger diverse behaviors, including unexpected accept, semantic reject, decoder failure, and timeout-retry.

Future Work

- Promote more modeled families from planned-only to live execution
- Generalize execution for any modeled NAS field
- Extend selected procedure-aware testing to more message flows
- Broaden coverage into richer NGAP and 5GSM mutation scenarios
- Explore direct decoder-level fuzzing as a complementary white-box extension

References

- [1] **RANsacked**: [A Domain-Informed Approach for Fuzzing LTE and 5G RAN-Core Interfaces](#)
- [2] **Optimizing 5G Protocol Fuzzing**: [Structure-Aware Techniques and Feedback Integration, ITASEC 2025](#)
- [3] **Open5GS**: [Open-source 5G Core and EPC implementation](#)
- [4] **UERANSIM**: [Open-source 5G UE and gNB simulator](#)
- [5] **AFL++**: [Advanced coverage-guided fuzzing framework](#)
- [6] **LibFuzzer**: [LLVM in-process coverage-guided fuzzing engine](#)
- [7] **Project repository**:
<https://github.com/omya3/TWIN-5G-fuzzing/tree/post-midterm-generalization>
 - **Github Branch**: **post-midterm-generalization**

Declaration on LLM Usage

LLMs Used

- OpenAI Codex / ChatGPT for coding assistance, debugging support, and writing refinement

Typical Prompt Types

- Understanding telecom fuzzing concepts and protocol-aware mutation ideas
- Debugging implementation issues in Python and C proxy logic
- Improving documentation, report wording, and slide structure
- Refining test workflows, campaign summaries, and presentation content

Thankyou

Limitations and Future Work

- The current implementation is strongest for supported live proxy-executable NAS families, not for every modeled family.
- Some modeled Identity Response families remain planned-only, such as length inconsistency and wrong-state delivery.
- Broader NGAP mutation coverage and wider 5GSM / post-registration expansion are still future work.
- Execution is not yet fully generic for any modeled NAS field across all messages.

Contd..

- Promote more modeled families from planned-only to live execution
- Generalize execution for any modeled NAS field
- Extend state-aware testing across more procedures
- Broaden beyond NAS-focused mutation into richer NGAP and 5GSM coverage

Conclusion

- TWiN implements a trace-driven, schema-aware NAS fuzzing workflow for 5G control-plane testing.
- The system supports live proxy execution, campaign planning, classification, recording, and report generation.
- The final evaluation reached 5 modeled NAS messages, 24 mutation families, and 97 recorded observations.
- Results show that structured malformed NAS inputs can trigger diverse behaviors, including unexpected accept, semantic reject, decoder failure, and timeout-retry.